

OOP in Python — First Principles Notes

Built concept by concept through active recall. No passive reading. Each concept answers one design question about how to organize software.

Table of Contents

1. [Why OOP Exists](#)
 2. [Class and Object](#)
 3. [Constructor and `self`](#)
 4. [Encapsulation](#)
 5. [Static Variables and Methods](#)
 6. [Collection of Objects](#)
 7. [Aggregation and Composition](#)
 8. [Inheritance, MRO, and Polymorphism](#)
-

Concept 1 — Why OOP Exists

Design question: *How do we organize large programs so that data has a clear owner?*

Before OOP

Programming was mostly **procedural** — data and functions were separate.

```
balance = 100

deposit(balance)
withdraw(balance)
```

This works well for small programs, algorithms, and utilities. **Procedural programming is not wrong.**

The Real Problem

As software became huge:

- Thousands of functions existed across many files
- Many modules modified the same data
- Bugs became difficult to trace — who changed what?
- Relationships between data were implicit (e.g. matching array indices)

The challenge wasn't writing code. It was **managing complexity**.

Could Good Naming Solve This?

Partially. Good naming improves readability. But it **doesn't enforce rules**. Anyone can still modify shared data if the language allows it. Discipline breaks down at scale.

OOP's Core Idea

Not: *"Put functions inside classes."*

Instead: **Give every piece of data an owner.**

```
Alice.balance
Bob.balance
John.balance
```

Each balance belongs to exactly one account. No ambiguity about who is responsible.

"Everyone Can Touch the Data" — What This Actually Means

"Everyone" doesn't mean every user. It means **every part of the program**.

```
withdraw.py
deposit.py
interest.py
loan.py
report.py
```

Without clear ownership, any of these modules can freely modify the same state. OOP provides language-level boundaries to prevent this.

Procedural vs OOP — The Real Difference

```
withdraw(account) # procedural — account is passive data
account.withdraw() # OOP — account is responsible for itself
```

Both produce the same result. The difference is **responsibility and ownership**, not syntax.

Why experienced programmers preferred `account.withdraw()` :

The account is responsible for changing its own balance.

When debugging, instead of asking "which function changes this data?" you ask "which object owns this data?" — this dramatically reduces the search space.

Was OOP Necessary?

No. Not everywhere.

Large, successful software like Linux and Git are mostly procedural. Good procedural design also scales. OOP simply gives **language-level support** for expressing ownership, responsibility, and boundaries.

Problem dominated by	Better fit
Algorithms, data transformations, hardware	Procedural
Long-lived entities, changing business rules, relationships	OOP

One-Liner

OOP wasn't invented because functions were insufficient. It was invented to organize large systems by giving data a clear owner and defining who is responsible for changing it.

Concept 2 — Class and Object

Design question: *What is this thing, and how do we create many of them consistently?*

Before Classes

Programmers already grouped related data using dictionaries:

```
dog = {  
    "name": "Buddy",  
    "age": 3,  
    "breed": "Labrador"  
}  
  
def bark(dog):  
    print(f"{dog['name']} says Woof!")
```

This worked. **Classes were not invented because programmers couldn't group data.**

The Limitation

Every programmer had to remember:

- Required fields and expected structure
- Which functions belong to which data
- Conventions that the language couldn't enforce

The language only knew this was a `dict`. It couldn't express: **"This is a Dog."**

Nothing stopped someone from creating `{"name": "Buddy", "ag": 3}` — a typo, no error, silent bug.

What a Class Solves

A class defines **once**:

- What data every object of this type has
- What behavior every object of this type has

```
class Dog:  
    def __init__(self, name, age, breed):  
        self.name = name  
        self.age = age  
        self.breed = breed  
  
    def bark(self):  
        print(f"{self.name} says Woof!")
```

The class is a **reusable definition** — not an actual dog. A blueprint, not a house.

What is an Object?

An object is one actual instance created from the class.

```
dog1 = Dog("Buddy", 3, "Labrador")  
dog2 = Dog("Max", 5, "Beagle")
```

Each has its own independent data. Both share the same methods.

Class vs Object

Class	Object
Definition	Actual instance
Exists once	Many can exist
Stores shared methods	Stores individual data
Describes what a Dog is	Represents one specific Dog

Memory Model

There is **one class** in memory. Objects point to it — methods are not copied into every object.

```

Dog Class
-----
__init__()
bark()

dog1 object      dog2 object
-----
name = Buddy     name = Max
age = 3          age = 5
breed = Labrador breed = Beagle
↓               ↓
→ Dog Class      → Dog Class

```

`bark()` is shared. Stored once. Called by all objects.

What Python Actually Does — Step by Step

```
dog = Dog("Buddy", 3, "Labrador")
```

1. Python finds the `Dog` class
2. Allocates memory for a new empty object
3. Calls `__init__(self, "Buddy", 3, "Labrador")` — where `self` is the new object
4. `__init__` fills in the object's data (`self.name` , `self.age` , etc.)
5. Returns the object
6. `dog` stores a **reference** to that object

Variables are References — Not Objects

```

dog = Dog("Buddy", 3, "Labrador")
another = dog
another.name = "Max"
print(dog.name)  # Max

```

`dog` and `another` are **references** pointing to the same object. Changing through either reference changes the same object.

```

dog ———|
      |
      ▼
    Dog Object ← one object in memory
      ▲
another—|

```

Why Not Just a Dictionary Factory?

```

def make_dog(name, age):
    return {"name": name, "age": age}

```

This is reusable. So why classes?

1. **A real type** — `type(dog)` returns `Dog`, not `dict`. This is what makes `isinstance()`, inheritance, and polymorphism possible. Not cosmetic — foundational.
2. **Shared methods** — stored once in the class, not duplicated per object
3. **Encapsulation** — data and behavior belong together, attached to the type
4. **Language-level structure** — responsibilities attached to the type, not floating functions

One-Liner

A class is a reusable definition of a new type. An object is one concrete instance of that type — with its own state, sharing the class's behavior.

Concept 3 — Constructor and `self`

Design question: How is an object actually created, and how does shared behavior know which object it's working on?

The Biggest Misconception

Most tutorials say: "`__init__` is the constructor."

Strictly speaking, that's wrong.

Python separates object creation into two distinct steps:

1. **Create** the object
2. **Initialize** the object

These are two different methods with two different responsibilities.

`__new__` VS `__init__`

Method	Responsibility
<code>__new__()</code>	Creates a new empty object in memory
<code>__init__()</code>	Fills that object with initial data

Think of building a house:

- `__new__` = constructing the walls and rooms. The house now exists.
- `__init__` = adding furniture, painting walls, moving in.

The house must exist before you can furnish it.

What Python Actually Does — Step by Step

```
dog = Dog("Buddy", 3)
```

Looks like one operation. Internally:

Step 1 — `__new__` is called:

```
Dog.__new__(Dog)
```

"Dog class, create one Dog object."

Result: an empty object exists in memory.

```
Dog Object
name = ?
age  = ?
```

Step 2 — `__init__` is called:

```
Dog.__init__(new_object, "Buddy", 3)
```

`self.name = "Buddy"` and `self.age = 3` fill in the object.

```
Dog Object
name = Buddy
age  = 3
```

Step 3 — Object is returned. `dog` now holds a reference to it.

Why Does `__new__` Take `Dog` Twice?

```
Dog.__new__(Dog)
```

This confuses everyone. The two `Dog`s have different jobs:

- First `Dog.` — which class's creation method to use
- Second `(Dog)` — what type of object to create

Think of it like:

```
Bakery.make(Bread)
```

Bakery = who does the work. Bread = what gets created.

The second `Dog` is **not** `self`. It is the class object itself. `self` only appears later, inside instance methods.

What is `self`? — The Real Explanation

Forget the definition. Understand the problem first.

```
buddy = Dog("Buddy", 3)
max    = Dog("Max", 5)
```

Now imagine:

```
def introduce():
    print(name) # which name? Buddy or Max?
```

The function has no idea which object it's working on. This is the problem `self` solves.

How Python Secretly Helps

When you write:

```
buddy.introduce()
```

Python internally converts it to:

```
Dog.introduce(buddy)
```

And:

```
max.introduce()
```

becomes:

```
Dog.introduce(max)
```

The object is automatically passed as the first argument. That first parameter is called `self` — but it's just a convention. You could call it anything:

```
def bark(self):
    print(self.name)

# identical to:
def bark(current_dog):
    print(current_dog.name)
```

`self` is not magic. It is just a normal parameter that Python fills in automatically.

Why Every Method Needs `self`


```
class Dog:
    def bark():          # no self
        print("Woof")

dog = Dog()
dog.bark()              # TypeError
```

Python converts `dog.bark()` → `Dog.bark(dog)` .

But `bark()` expects 0 arguments. Python is passing 1. Hence:

```
TypeError: bark() takes 0 positional arguments but 1 was given
```

The method has no idea which object called it — so it crashes.

The Biggest Mental Model Shift

Stop thinking:

```
dog.bark()
```

Start thinking:

```
Dog.bark(dog)
```

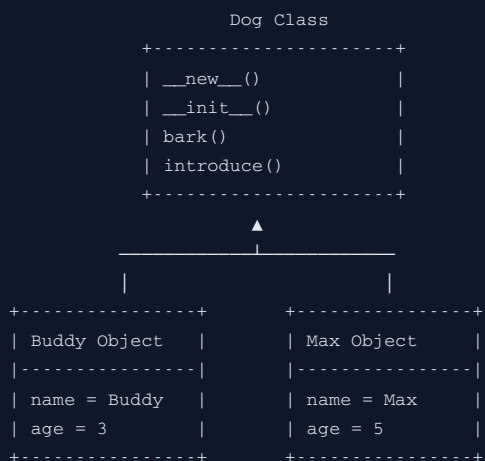
They are identical. The dot notation is just a convenience that Python translates automatically.

`self` Changes Every Call

```
buddy.introduce() → Dog.introduce(buddy) → self == buddy
max.introduce()  → Dog.introduce(max)   → self == max
```

The code inside `introduce()` never changes. **Only `self` changes.** This is how one method can correctly operate on millions of different objects.

Memory Visualization



Methods live **once** in the class. Data lives **separately** in every object. `self` is the bridge that connects a method call to the right object.

One-Liner

A class is a collection of shared behavior. An object is a piece of memory containing its own data. `self` is simply Python telling that shared behavior — "this is the object you're working on right now."

Concept 4 — Encapsulation

Design question: *Who is responsible for protecting an object's state?*

Why Was Encapsulation Invented?

Imagine a payroll system. An employee has a salary. Now imagine every module in the system can freely modify it:

```
class Employee:
    def __init__(self):
        self.salary = 50000
```

```
attendance.py → employee.salary += 1000
bonus.py      → employee.salary += 5000
tax.py        → employee.salary -= 12000
buggy_module.py → employee.salary = -500 # nothing stops this
```

Payroll crashes. Which module caused it? **You don't know.** Too many places can modify salary.

The Real Problem

The problem isn't that salary became negative. The real problem is:

Nobody owns the rules.

Who decides that salary cannot be negative? That bonuses have limits? That every change must be logged? If everyone can modify salary, nobody owns its correctness.

Encapsulation's Core Idea

Instead of: *"Everyone changes salary."*

Say: *"Only Employee changes salary."*

Outside code asks:

```
employee.give_raise(5000) # ✓ controlled
```

instead of:

```
employee.salary += 5000 # ✗ uncontrolled
```

The object becomes the **gatekeeper** of its own state.

Real Definition

Most books say: *"Bundle data and methods together."*

Better definition:

Encapsulation is controlling how an object's state can be accessed or modified.

The important word is **control**.

Without vs With Encapsulation

Without:

```
class BankAccount:
    def __init__(self):
        self.balance = 1000

account.balance = -999999 # no checks, no rules
```

With:

```
class BankAccount:
    def __init__(self):
        self._balance = 1000

    def withdraw(self, amount):
        if amount > self._balance:
            raise ValueError("Insufficient funds")
        self._balance -= amount

account.withdraw(100) # rules enforced inside
```

The rules live inside `withdraw()`. Not scattered across five modules.

Python Privacy — `_` vs `__`

Single underscore `_name` :

Python does **nothing**. Stored exactly as `_balance`. Pure convention meaning *"internal use — please don't touch directly."* The language won't stop you.

Double underscore `__name` :

Python performs **name mangling**. Internally renames `self.__balance` to `_BankAccount__balance`. This discourages accidental access and prevents name collisions in subclasses. It is **not** true privacy.

Name Mangling — The Surprising Example

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # stored as _BankAccount__balance

acc = BankAccount(100)
acc.__balance = 500 # creates a NEW attribute literally named __balance
print(acc.__balance) # 500 ← the new attribute
print(acc._BankAccount__balance) # 100 ← the original, untouched
```

Memory now contains **two separate attributes**:

```
_BankAccount__balance = 100 ← original
__balance = 500 ← new, created outside the class
```

Name mangling outside the class does not apply. This surprises almost everyone the first time.

Python vs Java

Java: `private double balance;` — compiler enforces access. `account.balance = 100` is a compile error.

Python: Relies on conventions and trust. `_balance` and `__balance` signal intent, but a determined programmer can always get through.

Python's philosophy: *"We're all consenting adults."*

Both languages still agree on the important idea: **the object owns the rules**.

Why Encapsulation Still Matters Even Without Enforcement

Today `deposit()` simply adds money. Tomorrow business says every deposit must:

- Log the transaction
- Send an SMS notification
- Update reward points
- Detect fraud
- Reject deposits over ₹1,00,000

Where do you change the code? **Only inside** `deposit()` .

Every caller automatically follows the new rules. Nobody else's code changes. This is the real payoff.

The Deepest Insight

Encapsulation does not protect **variables**.

It protects **invariants** — things that must always remain true:

```
Balance ≥ 0
Age ≥ 0
Salary ≥ Minimum Wage
Inventory ≥ 0
```

Only the object should ensure these rules hold. If anyone can reach in and change the data directly, invariants can be broken silently.

Memory Visualization

Without encapsulation:

```
Attendance ─┐
Tax ─────────┤
Bonus ─────────┤→ Employee.salary (anyone modifies)
Payroll ─────┤
HR ───────────┘
```

With encapsulation:

```
Attendance ─┐
Bonus ───────┤
HR ───────────┤→ Employee.give_raise()
Payroll ─────┤
               |
               ▼
            salary (only Employee modifies)
```

One-Liner

Encapsulation is not about hiding data — it's about ensuring the object that owns the data is the only place responsible for keeping that data valid.

Concept 5 — Static Variables and Methods

Design question: *What do we do when data or behavior belongs to the class itself, not to any one object?*

The Problem That Creates the Need

Every object has its own data:

```
class Dog:
    def __init__(self, name):
        self.name = name

buddy = Dog("Buddy")
max = Dog("Max")
```

Now suppose the vet clinic asks: **"How many dogs have we created in total?"**

The wrong approach:

```
class Dog:
    def __init__(self, name):
        self.name = name
        self.count = 0  # ✗ every dog gets its own count = 0
```

Each object has its own `count`. There is **no total**. This data doesn't belong to one dog — it belongs to the whole concept of "Dog."

First Principle: Who Owns This?

If it belongs to **one object** → instance variable. If it belongs to **all objects together** → class variable.

Instance Variable vs Class Variable

Instance variable — lives separately inside every object:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

Class variable — lives once, inside the class itself:

```
class Employee:
    company = "Atlas"
```

```

      Employee Class
+-----+
| company = Atlas | ← one copy, shared
+-----+
      ▲
      |
+-----+-----+
| Alice Object | | Bob Object |
| name = Alice | | name = Bob  |
| salary = 50000 | | salary = 60000 |
+-----+-----+
```

Python's Lookup Order

```
print(emp.company)
```

1. Does `emp` (the object) have `company`? → No

2. Does the `Employee` class have `company`? → Yes → return it

The Shadowing Trap (Classic Interview Question)

```
class Employee:
    company = "Atlas"

e1 = Employee()
e2 = Employee()

e1.company = "Acme"

print(e1.company)      # Acme
print(e2.company)      # Atlas
print(Employee.company) # Atlas
```

Why: `e1.company = "Acme"` does **not** modify the class variable. It creates a **new instance variable** on `e1` that shadows the class variable for that object only.

```
Employee Class:  company = Atlas
e1:              company = Acme  ← new instance variable, shadows class
e2:              (nothing)       ← still falls back to class
```

`e2` and `Employee.company` are untouched.

Class vs Instance Variable — Summary

Instance Variable	Class Variable
Lives inside each object	Lives once, inside the class
Different for every object	Shared by all objects
Accessed with <code>self</code>	Accessed via <code>ClassName</code> or <code>cls</code>
Changes affect one object	Changes affect all (unless shadowed)

Static Methods

Not every function needs an object. A static method needs **neither** `self` nor `cls` — it's a plain function that's logically grouped inside the class.

```
class Calculator:
    @staticmethod
    def add(a, b):
        return a + b
```

Should everything be a static method? No. If a function doesn't logically belong to the class's concept, a normal standalone function is often better than forcing it into the class.

Good use case — clearly belongs to the concept, needs nothing:

```
class Fraction:
    @staticmethod
    def is_valid_denominator(d):
        return d != 0
```

Class Methods and `cls`

Just as `self` refers to the current **object**, `cls` refers to the current **class**.

```
class Dog:
    count = 0

    @classmethod
    def total(cls):
        return cls.count
```

`Dog.total()` conceptually becomes `Dog.total(Dog)` — inside, `cls == Dog`.

Factory Methods — Why `cls(...)` Beats Hardcoding

Parsing `"John,50000"` into an `Employee` object:

```
class Employee:
    @classmethod
    def from_string(cls, data):
        name, salary = data.split(",")
        return cls(name, int(salary)) # ✓ not Employee(...)
```

Why `cls(...)` and not `Employee(...)`?

If later:

```
class Manager(Employee):
    pass
```

Then `Manager.from_string("Alice,80000")`:

- Using `cls(...)` → returns a `Manager` ✓
- Using `Employee(...)` → would incorrectly return an `Employee`, even when called on `Manager` ✗

`cls` always refers to whichever class actually called the method — this is what makes factory methods reusable across subclasses.

Three Types of Methods — Decision Framework

```
Does it need object data (self)?
YES → Instance Method
NO ↓
Does it need class data (cls)?
YES → Class Method
NO ↓
Does it logically belong to this class?
YES → Static Method
NO → Just write a normal function
```

Concept	Belongs To	Stored In	Receives
Instance Variable	One object	Object	—
Class Variable	Entire class	Class	—
Instance Method	One object	Class	<code>self</code>
Class Method	Entire class	Class	<code>cls</code>
Static Method	Neither	Class	Nothing

One-Liner

Every OOP design decision comes down to: who owns this data or behavior? One object → `self`. The class itself → `cls`. Neither, but conceptually related → static method (or often just a normal function).

Concept 6 — Collection of Objects

Design question: *How do we manage many objects together, and who owns the group?*

The Problem

A single object only knows about **itself**.

```
employee = Employee("Alice", 50000, "HR")
```

This object can answer *"what's my salary?"* but **cannot** answer:

- What's the company's total payroll?
- How many employees work in HR?
- Who earns the highest salary?

These questions need **many objects together**.

The Idea — A Collection of Objects

```
alice = Employee("Alice", 50000, "HR")
bob = Employee("Bob", 60000, "IT")
charlie = Employee("Charlie", 70000, "HR")

employees = [alice, bob, charlie]
```

The list stores **references**, not copies:



Who Should Own the Collection?

Same golden question as always: **who owns the employees?**

Not Alice, not Bob — the **Company**.

```
class Company:
    def __init__(self, name):
        self.name = name
        self.employees = []
```

Why Not a Global List?

```
employees = []    # ✕ global

atlas = Company("Atlas")
google = Company("Google")
```

Both companies would share the same list. Which employees belong to which company? Impossible to know.

Instead, each `Company` object owns its own `employees` list — same ownership principle from Concept 1 and 4.

What a Collection Enables

Aggregation:

```
def total_payroll(self):
    return sum(emp.salary for emp in self.employees)
```

Filtering:

```
def employees_in_department(self, dept):
    return [emp for emp in self.employees if emp.department == dept]
```

Counting:

```
len(self.employees)
```

A single object can never do any of this — these operations require seeing all objects at once.

Division of Responsibility

Object	Responsible for
Employee	its own name, salary, department
Company	the collection, payroll, searching, filtering, counting

Collections Store References — Not Copies

```
emp = company.employees[0]
emp.salary = 99999
print(company.employees[0].salary) # 99999
```

There was never a copy — only one `Employee` object, with two references (`emp` and `company.employees[0]`) pointing to it. Changing through either reference changes the same underlying object. This is the exact same reference model from Concept 2, now applied inside a list.

One-Liner

A collection of objects models a "one-to-many" relationship. Individual objects manage their own state, while the owning object (like `Company`) manages the group and performs operations that require seeing all of them together.

Concept 7 — Aggregation and Composition

Design question: *When one object contains another, who owns the contained object's lifetime?*

The Problem

Designing a `Car`:

```
class Car:
    def __init__(self):
        self.brand = ...
        self.engine_cc = ...
        self.horsepower = ...
        self.fuel_type = ...
        self.cylinders = ...
```

This works, but ask: **is an engine just a few variables?** No. An engine has its own data, its own behavior, its own identity. This tells us: **Engine deserves to be its own object.**

First Principle

Does this thing have its own state and behavior? If yes — make it a separate class.

Engine as a Separate Object

```
class Engine:
    def __init__(self, horsepower, fuel_type):
        self.horsepower = horsepower
        self.fuel_type = fuel_type

class Car:
    def __init__(self, model, engine):
        self.model = model
        self.engine = engine    # Car HAS-A Engine
```

`Car` doesn't contain `horsepower`, `fuel_type`, etc. directly — it contains an **Engine object**.

"Has-a" vs "Is-a"

Has-a — one object contains another:

```
Car has an Engine
Company has Employees
Library has Books
```

Is-a — one object is a specialized version of another:

```
Dog is an Animal
Manager is an Employee
```

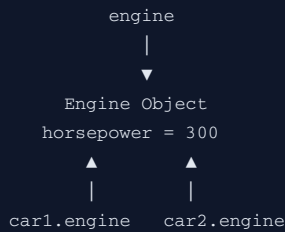
This is **Inheritance**, not Aggregation. Completely different relationship.

Why Not Cram Everything Into Car?

If `Truck`, `Motorcycle`, and `Generator` all need engine info, and engine fields live inside every class, you duplicate `horsepower`, `fuel_type`, `serial_number` everywhere. Instead, all of them reuse one `Engine` class — one class, many users.

Memory Model

```
engine = Engine(300)
car1 = Car("Model X", engine)
car2 = Car("Model Y", engine)
```



One `Engine`, multiple references. Changing `engine.horsepower = 400` updates the value seen by both cars — same reference principle as always.

Why Aggregation Matters

1. **Separation of responsibilities** — Car handles car logic, Engine handles engine logic
2. **Reusability** — the same Engine class serves Car, Truck, Bike, Generator without duplication
3. **Better organization** — small, focused classes instead of one giant class

Aggregation — Definition

One object **uses** another object, but the contained object has an **independent lifecycle**.

```
engine = Engine() # created independently, outside Car
car = Car(engine) # passed in
```

The engine existed before the car. Deleting the car doesn't delete the engine — other references can still point to it.

Composition — Definition

```
class Car:
    def __init__(self):
        self.engine = Engine() # created INSIDE Car
```

The car creates its own engine. The engine's lifecycle is tied to the car — if nothing else references it, it disappears along with the car (note: in Python, `del` removes a *reference*, not the object itself — the object is garbage-collected once its reference count hits zero).

Aggregation vs Composition — Side by Side

	Aggregation	Composition
Relationship	Has-a	Has-a
Lifecycle	Independent	Owned by container
Object created outside?	✓ Yes	✗ No, created inside
Object survives owner?	✓ Usually yes	✗ Usually no
Coupling	Loose	Strong

Real-World Examples

Aggregation (object survives independently):

```
Company → Employees (employees can resign, survive, join elsewhere)
Library → Books      (books can move to another library)
Team    → Players     (players can change teams)
```

Composition (object's lifecycle is tied to the owner):

```
House → Rooms      (rooms belong to one house)
Book  → Pages       (pages don't exist independently in this model)
Human → Heart        (heart's lifecycle belongs to the human)
```

The Design Question

Can this object exist meaningfully on its own?

Yes → Aggregation. No → Composition.

One-Liner

Aggregation means "I use this object." Composition means "I own this object." Both are "has-a" relationships — the real difference is who owns the contained object's lifetime.

Concept 8 — Inheritance, MRO, and Polymorphism

Design question: *What do we do when multiple classes share common behavior? And can different objects respond to the same request in their own way?*

Why Inheritance Exists

Without it, building a zoo means duplicating `name`, `eat()`, `sleep()` across `Dog`, `Cat`, `Horse`, `Cow`. The problem isn't typing — it's **maintenance**. If `eat()` changes, you update every animal class. Same bug, five places.

The fix: ask what's common, put it in a parent class.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def eat(self):
        print("Eating")

    def sleep(self):
        print("Sleeping")

class Dog(Animal):
    pass

class Cat(Animal):
    pass
```

Both get `name`, `eat()`, `sleep()` without rewriting anything.

Inheritance models "is-a": `Dog IS-A Animal`. This is different from Aggregation's "has-a" (`Car HAS-A Engine`) — two completely separate relationships.

What Actually Happens in Memory

Common misconception: methods get **copied** from parent to child. **Wrong.**

```
Animal Class
eat()
sleep()
  ▲
  |
Dog Class
  ▲
  |
dog object
```

Dog only knows its parent. Calling `dog.eat()` triggers a **dynamic lookup**:

1. Does `Dog` have `eat()`? No.
2. Ask `Animal`. Found it.
3. Execute whatever `Animal.eat()` currently is.

Consequence: if you redefine `Animal.eat = new_function` after `Dog` objects already exist, `dog.eat()` immediately uses the new version. Lookup happens every call — not once at class creation. Inheritance is based on **lookup, not duplication**.

`super().__init__()` — Why It Exists

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        self.breed = breed    # ✗ Animal.__init__ never runs
```

```
d = Dog("Buddy", "Labrador")
print(d.name)    # AttributeError — name was never set
```

Only `Dog.__init__()` ran. `self.name = name` inside `Animal.__init__` never executed, because nothing called it.

Fix:

```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)    # let Animal initialize its own part
        self.breed = breed
```

Principle: each class is responsible for initializing its own state. `super()` lets every class in the chain do its job.

Overriding vs Extending

Overriding — child replaces parent's version entirely:

```
class Dog(Animal):
    def speak(self):
        print("Woof")    # Animal's version is ignored
```

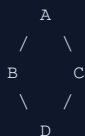
Extending — child reuses parent's version, then adds more:

```
class Dog(Animal):
    def eat(self):
        super().eat()
        print("Wagging tail")
# Output: Eating / Wagging tail
```

Types of Inheritance

Single:	Animal → Dog
Multilevel:	Animal → Mammal → Dog
Hierarchical:	Animal → Dog, Cat, Horse (one parent, many children)
Multiple:	Flying + Mammal → Bat (Python supports this; Java doesn't for classes)

The Diamond Problem and MRO



If every class defines `greet()`, which one does `d.greet()` call? Without rules — ambiguous. This is the **diamond problem**.

Python solves it by computing one deterministic lookup order — the **MRO (Method Resolution Order)** — using **C3 linearization**. You don't need to memorize the algorithm, just its guarantees:

- Every class appears exactly once
- Parent-before-grandparent relationships stay consistent

- Left-to-right declared order is respected where possible

For `class D(B, C)`, the MRO is `D → B → C → A → object`. Calling `d.greet()` walks this list and stops at the **first match** — `D` has none, `B` does, so `B` wins. Inspect any class's order with `ClassName.mro()`.

Polymorphism

```
animals = [Dog(), Cat(), Cow()]
for a in animals:
    a.speak()
# Woof / Meow / Moo
```

Same line of code (`a.speak()`), different behavior — determined by the **actual object's type**, not by the code calling it.

TV remote analogy: one "Power" button. Samsung remote turns on a Samsung. LG remote turns on an LG. Same command, different implementation underneath.

Core principle: the caller doesn't care which subclass it has. It just says `speak()`. Each object decides how. This is called **dynamic dispatch** — the decision of *which* method runs happens at **runtime**, based on the object's real type.

Method Overloading vs Operator Overloading

Java-style method overloading (same name, different parameter lists, resolved at **compile time** based on argument types/count) — **Python does not support this:**

```
def add(a):
    ...
def add(a, b):
    ... # this simply replaces the first definition
```

Python resolves functions at runtime by name only. Instead, Python uses default arguments, `*args`, keyword arguments, or type checks inside one function.

Operator overloading — Python *does* support this, through dunder methods:

```
f1 + f2      → f1.__add__(f2)
f1 == f2     → f1.__eq__(f2)
f1 < f2      → f1.__lt__(f2)
print(f1)    → f1.__str__()
```

You teach Python how your custom object should respond to built-in operators.

Polymorphism vs Overloading — The Precise Distinction

	Polymorphism	Method Overloading
What varies	Same method call, different object types	Same method name, different parameter lists
Decided by	The object's actual type	The arguments (count/type)
When decided	Runtime	Compile-time (Java)
Mechanism	Inheritance + overriding	Different signatures
Python support	Fully supported	Not supported in traditional form

They both let "one name behave differently" — but polymorphism is decided by **what the object is**, at **runtime**; overloading is decided by **what arguments are passed**, at **definition/compile time**.

One-Liner

Inheritance shares common behavior through dynamic lookup, not copying. `super()` lets every class in the chain initialize its own state. MRO gives a deterministic order for resolving the diamond problem. Polymorphism lets the same method call produce different behavior based on the actual object's type — decided at runtime, not by the caller.

Master Summary

The OOP Design Questions

Class	→ What is this thing?
Object	→ Which specific thing?
Encapsulation	→ Who protects this state?
Aggregation/Composition	→ What other things does it have?
Inheritance	→ What common behavior can be reused?
Polymorphism	→ Can different objects respond to the same request, differently?

Everything in One Place

- **Procedural programming** separates data and functions. Works well for algorithms and utilities.
- **OOP** organizes software around **ownership and responsibility**.
- A **class** is a reusable definition of a new type. An **object** is one actual instance of it.
- The **class stores shared methods**. The **object stores its own data**.
- `__new__()` **creates** an empty object in memory. `__init__()` **fills** it with data. They are separate steps.
- `__init__` is the initializer, not the constructor. `__new__` is the real constructor.
- `dog.method(x)` is identical to `Dog.method(dog, x)` — Python translates automatically.
- `self` is just the object receiving the method call. Not magic — a normal parameter Python fills in.
- The same method is shared by all objects. Only `self` changes per call.
- Without `self`, a method has no way to know which object called it → `TypeError`.
- Variables like `dog` hold **references**, not the objects themselves. Multiple variables can point to the same object.
- **Encapsulation** = controlling how an object's state is accessed or modified. The object is the gatekeeper.
- `__name` = convention only. Python does nothing. `__name` = name mangling → `__ClassName__name`. Not true privacy.
- Name mangling outside the class doesn't apply — `acc.__balance = 500` creates a new attribute, doesn't touch the original.
- Encapsulation protects **invariants**, not secrets. The real benefit: all rules for changing state live in one place.
- Python trusts programmers. Java enforces access. Both agree: the object owns the rules.
- **Class variable** lives once, inside the class — shared by all objects. **Instance variable** lives separately in each object.
- `e1.company = "Acme"` doesn't change the class variable — it creates a new instance variable that *shadows* it for `e1` only.
- **Static method** (`@staticmethod`) needs neither `self` nor `cls` — a plain function grouped inside the class for organization.
- **Class method** (`@classmethod`) receives `cls` — used for factory methods. `cls(...)` instead of hardcoding `ClassName(...)` makes the method subclass-safe.
- Decision rule: needs object data → instance method. Needs class data → class method. Needs neither → static method or a plain function.
- A **collection of objects** (e.g. `Company.employees`) stores **references**, not copies — same reference model from Concept 2.
- The object that logically owns the group should own the collection (Company owns employees, not a global list).
- Collections enable aggregation, filtering, searching, counting — operations a single object can never do alone.
- **"Has-a"** (Aggregation/Composition) ≠ **"is-a"** (Inheritance). Different relationships entirely.
- **Aggregation**: contained object created outside, independent lifecycle, survives the owner (Company → Employees).
- **Composition**: contained object created inside the owner, lifecycle tied to it (Car → Engine created in `__init__`).
- Design question: can this object exist meaningfully on its own? Yes → Aggregation. No → Composition.
- **Inheritance** shares behavior via dynamic lookup, not copying — modifying a parent method after child objects exist still affects them.
- `super().__init__()` ensures every class in the chain initializes its own state. Skipping it leaves the parent's attributes missing.

- **Overriding** replaces parent behavior. **Extending** calls `super()` then adds more.
- **MRO** (via C3 linearization) gives a deterministic lookup order for the diamond problem — every class appears once, declared left-to-right order is respected.
- **Polymorphism**: same method call, different behavior, decided by the object's actual type, at runtime.
- **Method overloading** (Java-style) $\neq$ polymorphism. Overloading is decided by arguments at definition time; Python doesn't support it. **Operator overloading** (`__add__`, `__eq__`, etc.) is supported.

The Core Mental Model

OOP is not about "functions inside a class." It's about creating objects that own their state, know their behavior, and are responsible for maintaining themselves. It organizes relationships between them, reuses common behavior through inheritance, and allows different objects to respond to the same message in their own way through polymorphism.

Interview Questions

Concept 1 — Why OOP Exists

Q1. What problem does OOP solve that procedural programming doesn't?

OOP gives language-level support for data ownership and access control. Procedural programming can achieve similar organization through discipline, but OOP enforces it structurally.

Q2. If OOP is so useful, why are systems like Linux and Git mostly procedural?

Because their problem domain is dominated by algorithms, data transformations, and hardware interaction — where procedural code is often simpler. OOP fits problems with long-lived entities, changing business rules, and complex relationships. The paradigm should match the domain.

Q3. What is the real difference between `withdraw(account)` and `account.withdraw()` ?

Responsibility. The first treats the account as passive data operated on by external functions. The second says the account is responsible for managing its own state. When debugging, you ask "which object owns this?" instead of "which function touched this?" — reducing the search space.

Concept 2 — Class and Object

Q4. What does a class give you that a dictionary factory function doesn't?

A real type (enabling `isinstance`, inheritance, polymorphism), shared methods stored once in memory, encapsulation of data and behavior together, and language-level structure for defining responsibilities.

Q5. Are methods copied into every object?

No. Methods live once in the class. Objects store only their own data and a reference back to the class. When a method is called, Python looks it up in the class dynamically.

Q6. What does this print and why?

```
a = Dog("Buddy")
b = a
b.name = "Max"
print(a.name)
```

Max. Both `a` and `b` are references to the same object. Changing through either reference changes the same underlying object.

Concept 3 — Constructor and `self`

Q7. Is `__init__` the constructor?

No. `__init__` is the initializer. `__new__` is the actual constructor — it allocates memory and creates the empty object. `__init__` fills it with data afterward.

Q8. What happens if you define a method without `self` ?

```
class Dog:
    def bark():
        print("Woof")
Dog().bark()
```

TypeError. Python translates `dog.bark()` to `Dog.bark(dog)` — passing the object as the first argument. But `bark()` expects zero arguments, so it crashes.

Q9. How can one method work correctly for millions of different objects?

Because `self` changes every call. The code never changes — only `self` (the object being operated on) changes.
`buddy.bark()` → `self == buddy` , `max.bark()` → `self == max` .

Concept 4 — Encapsulation

Q10. What is the difference between `_name` and `__name` in Python?

`_name` is pure convention — Python does nothing. `__name` triggers name mangling: Python renames it to `__ClassName_name` , discouraging accidental access and preventing subclass name collisions. Neither provides true privacy.

Q11. What does this code print?

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

acc = BankAccount(100)
acc.__balance = 500
print(acc.__balance)
print(acc._BankAccount__balance)
```

500 then 100 . `acc.__balance = 500` creates a new attribute (name mangling doesn't apply outside the class). The original `_BankAccount__balance` remains 100 . Two separate attributes exist.

Q12. If Python doesn't enforce privacy, what's the point of encapsulation?

Encapsulation protects invariants, not secrets. The value is that all rules for modifying state live in one place. When business logic changes (logging, validation, fraud detection), you update one method — every caller automatically follows the new rules.

Concept 5 — Static Variables and Methods

Q13. What is the output?

```
class Employee:
    company = "Atlas"

e1 = Employee()
e2 = Employee()
e1.company = "Acme"

print(e1.company)
print(e2.company)
print(Employee.company)
```

`Acme`, `Atlas`, `Atlas`. `e1.company = "Acme"` creates a new instance variable on `e1` that shadows the class variable. `e2` has no instance variable, so it falls back to the class. The class variable itself is untouched.

Q14. When do you use `@staticmethod` vs `@classmethod`?

`@staticmethod` when the method needs neither `self` nor `cls` — it's a utility function that logically belongs to the class. `@classmethod` when it needs access to the class itself (e.g. factory methods). Use `cls(...)` instead of hardcoding `ClassName(...)` to support subclasses.

Q15. Why does `cls(...)` matter in a factory method?

If `Manager(Employee)` calls `Employee.from_string(...)`, using `cls(...)` creates a `Manager`. Hardcoding `Employee(...)` would always create an `Employee`, even when called on a subclass. `cls` makes the factory method inheritance-safe.

Concept 6 — Collection of Objects

Q16. Why should a collection of employees live inside a `Company` object rather than as a global list?

Because the employees belong to a specific company. A global list can't distinguish which employees belong to which company. Ownership: the `Company` owns its employees.

Q17. If you do `emp = company.employees[0]` and then `emp.salary = 99999`, does `company.employees[0].salary` change?

Yes. `emp` and `company.employees[0]` are references to the same object. There is no copy. Changing through either reference changes the same underlying object.

Concept 7 — Aggregation and Composition

Q18. What is the difference between Aggregation and Composition?

Both are "has-a" relationships. Aggregation: the contained object has an independent lifecycle (created outside, can survive the owner). Composition: the contained object's lifecycle is tied to the owner (created inside, typically destroyed with it).

Q19. Give an example of each.

Aggregation: `Company` and `Employees` — employees exist independently and can join another company.
Composition: `Car` and `Engine` created inside `__init__` — the engine has no independent purpose in this model.

Q20. When should you create a separate class for something instead of adding its fields to the parent?

When that "something" has its own state and behavior — its own identity. An `Engine` isn't just a few variables on a `Car`. It has its own data, its own methods, and can be reused by `Truck`, `Bike`, `Generator`.

Concept 8 — Inheritance, MRO, and Polymorphism

Q21. Does a child class copy parent methods?

No. It looks them up dynamically. If the parent method changes after objects exist, existing child objects automatically use the new version. Inheritance is lookup, not duplication.

Q22. What happens if you forget `super().__init__()` in a child class?

The parent's `__init__` never runs. Any attributes initialized there (like `self.name`) won't exist on the object. Accessing them raises `AttributeError`.

Q23. What is the MRO for `class D(B, C)` when B and C both inherit from A?

`D → B → C → A → object`. C3 linearization guarantees every class appears once, parent-before-grandparent relationships stay consistent, and the declared left-to-right order (B before C) is respected.

Q24. What is the difference between overriding and extending a parent method?

Overriding: the child completely replaces the parent's version. Extending: the child calls `super().method()` first to reuse the parent's behavior, then adds its own logic on top.

Q25. What is polymorphism?

The same method call producing different behavior depending on the object's actual type — decided at runtime through dynamic dispatch. `a.speak()` calls `Dog.speak()` if `a` is a Dog, `Cat.speak()` if `a` is a Cat. The caller doesn't care which subclass it has.

Q26. Are polymorphism and method overloading the same thing?

No. Polymorphism is decided by the object's type, at runtime. Method overloading is decided by argument types/count, at compile/definition time. Python fully supports polymorphism but does not support Java-style method overloading — the last function definition with a given name simply replaces earlier ones.

Q27. How does Python support operator overloading?

Through dunder methods: `+` → `__add__()`, `==` → `__eq__()`, `<` → `__lt__()`, `print()` → `__str__()`. You define these methods on your class to teach Python how your custom type should respond to built-in operators.

Cross-Cutting / Tricky Questions

Q28. Name the four pillars of OOP and give the core idea of each in one sentence.

Encapsulation: the object protects and manages its own state. Abstraction: users interact with what an object does, not how. Inheritance: share common behavior to avoid duplication ("is-a"). Polymorphism: the same message can produce different behavior depending on the actual object.

Q29. What is the difference between "is-a" and "has-a"?

"Is-a" = inheritance (Dog is-a Animal — shared behavior). "Has-a" = aggregation/composition (Car has-a Engine — contained object). Using the wrong one is a common design mistake.

Q30. You have `Dog`, `Cat`, `Parrot` classes. A function takes any animal and calls `animal.speak()`. What OOP concept makes this work?

Polymorphism. The function doesn't know or care about the specific subclass. It calls `speak()`, and each object's type determines which implementation runs — decided at runtime through dynamic dispatch.